TECHNICAL SPECIFICATION

V3.2 · 2026-04-17

LSIE-MLF

Technical Specification

Engineering Blueprint for Autonomous AI Implementation and Enterprise Deployment

Live Stream Inference Engine — Machine Learning Framework

CONTENT VERSION	SCHEMA VERSION	CLASSIFICATION	HARDENING BASELINE
v3.2	v3.0	CONFIDENTIAL	21-Rule Audit

CONTENTS

Table of contents

0 **Document control and canonical terminology** 3
- 0.1 Document purpose 3
- 0.2 Normative language 3
- 0.3 Canonical terminology registry 3
- 0.4 Version history 5

1 **System philosophy and operational scope** 5

2 **Deterministic system data flow pipeline** 6

3 **Codebase architecture and directory mapping** 8
- 3.1 Directory hierarchy 9
- 3.2 Build context rules 10

4 **Core architecture modules** 10
- 4.A Hardware & transport 10
- 4.B Ground truth ingestion 12
- 4.C Orchestration & synchronization 14
- 4.D Multimodal ML processing 17
- 4.E Experimentation & analytics 19
- 4.F Context enrichment 20

5 **Data governance and security** 21
- 5.1 Ephemeral vault specification 21
- 5.2 Data classification 22

6 **Interface contracts and payload schemas** 23
- 6.1 InferenceHandoffPayload — JSON schema draft 07 23
- 6.2 PhysiologicalSampleEvent — JSON schema draft 07 25
- 6.3 PhysiologicalContext — JSON schema draft 07 28

7 **Mathematical specifications** 29
- 7A AU12 facial action unit geometry 30
- 7B Thompson sampling reward pipeline 33
- 7C Physiological co-modulation analytics 36

8 **Deterministic LLM prompt specification** 40
- 8.1 Inference parameter constraints 41
- 8.2 Expected output schema 41

8.3 System prompt (canonical) . . . 42

9 Docker compose runtime topology 42

9.1 Container topology . . . 42
9.2 Volume mounts . . . 43
9.3 GPU allocation . . . 43
9.4 Device passthrough . . . 44
9.5 Network configuration . . . 44
9.6 Service dependency order . . . 44

10 Dependency and runtime matrix 45

10.1 System requirements . . . 45
10.2 Python package pinning . . . 45

11 Variable extraction matrix 46

12 Error handling and recovery specification 50

12.1 Network disconnection . . . 50
12.2 Hardware device loss . . . 50
12.3 Worker process crash . . . 51
12.4 Queue overload . . . 51

13 Autonomous implementation audit checklist 52

SECTION 0

Document control and canonical terminology

0.1 Document purpose

This document is the single authoritative specification for the Live Stream Inference Engine — Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete LSIE-MLF system using only this document. No external documentation is assumed or required.

0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

TERM	MEANING
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

0.3 Canonical terminology registry

Every system component has exactly one canonical name. All prior synonyms are retired. Implementations MUST use only the canonical identifiers listed below.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
API Server	`api`, `FastAPI server`, `web server`, `ASGI server`	The FastAPI application process that serves REST endpoints on port 8000. Container name: api.
ML Worker	`worker`, `Celery node`, `Celery worker`, `GPU worker`, `inference worker`	The Celery consumer process that executes all GPU-bound ML inference tasks. Container name: worker.
Message Broker	`Redis`, `queue`, `task queue`, `broker`	The Redis in-memory data store that brokers Celery task queues between the API Server and the ML Worker. Container name: redis.
Persistent Store	`postgres`, `PostgreSQL`, `database`, `relational database`, `DB`	The PostgreSQL relational database that persists analytical metrics and experiment state. Container name: postgres.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
Capture Container	`stream_scrcpy, scrcpy container, capture service, stream ingester`	The privileged Docker container responsible for physical USB I/O and raw audio-visual stream extraction. Container name: stream_scrcpy.
IPC Pipe	`FIFO, named pipe, POSIX pipe, audio pipe, kernel pipe, video pipe`	POSIX Named Pipes located at `/tmp/ipc/audio_stream.raw` (audio) and `/tmp/ipc/video_stream.mkv` (video) used for zero-copy data transfer between the Capture Container and the Orchestrator Container.
Ephemeral Vault	`24-hour vault, data vault, transient storage, secure buffer`	The AES-256-GCM encrypted transient storage regime for raw media buffers, subject to mandatory 24-hour secure deletion.
InferenceHandoffPayload	`handoff schema, payload schema, inference payload`	The JSON Schema Draft 07 contract governing data exchange between Module C and Module D, and from Module D to Module E.
Orchestrator Container	`orchestrator, segment producer, Module C worker`	Dedicated Docker container running the Module C orchestration loop. Uses the same image as the ML Worker but with a different CMD (`python3.11 -m services.worker.run_orchestrator`). Produces Celery tasks consumed by the ML Worker.
Physiological Chunk Event	`Physiological Sample Event, oura event, HRV event, wearable event, physio event`	A normalized wearable telemetry chunk record produced by the API Server hydration path and enqueued to Redis for the Orchestrator Container to drain. Defined by the PhysiologicalChunkEvent schema in §6.2.
Physiological Context	`physio context, HRV context, biometric context`	A per-subject derived snapshot of physiological state attached to each 30-second InferenceHandoffPayload segment via the _physiological_context dict key. The snapshot may include RMSSD, heart rate, validity_ratio, is_valid, freshness, staleness, source_kind, and derivation_method. Defined by the PhysiologicalContext schema in §6.3.
Physiological State Buffer	`physio buffer, HRV buffer, state cache`	An in-memory rolling buffer in the Orchestrator Container keyed by subject_role ('streamer' or 'operator'). Updated by draining Physiological Chunk Events from the physio:events Redis list and used to derive scalar physiological snapshots at segment assembly time.
Co-Modulation Index	`co-mod score, synchrony index, physiological correlation`	The Pearson correlation coefficient between 1-minute resampled, time-aligned valid streamer and operator RMSSD sequences over a rolling 10-minute analysis window. Computed by Module E and persisted to the comodulation_log table. Null when fewer than 4 paired observations are available or either aligned series has zero variance.

0.4 Version history

VERSION	DATE	DESCRIPTION
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.
v3.0	2026-04-01	Mathematical recipe (Thompson Sampling, continuous reward, fractional Beta-Bernoulli update), E2E gap fixes, hardening baseline, 6 spec amendments incorporated (cuDNN 8, Ubuntu 24.04, orchestrator container, scrcpy v3.3.4, audio chunk 1/30s, INT8 locked), 63 new tests, reward pipeline formalization.
v3.1	2026-04-15	Physiological co-modulation extension. Added Oura Ring webhook ingestion (§4.B.2), Orchestrator physiological state buffer (§4.C.4), Module E co-modulation analytics (§4.E.2), PhysiologicalSampleEvent and PhysiologicalContext schemas (§6.2, §6.3), RMSSD co-modulation math (§7C), physiological data governance, failure matrix entries, variable matrix entries, and audit checklist items. §7B Thompson Sampling reward pipeline explicitly unchanged.
v3.2	2026-04-17	Physiological ingestion model corrected. Oura webhook handling no longer assumes provider-delivered scalar 5-minute RMSSD payloads. Webhooks are treated as change notifications that trigger OAuth2-backed hydration, normalized physiological chunk transport, rolling RMSSD derivation in Module C, validity gating, and 1-minute co-modulation resampling. Operator dashboard stack migrated from Streamlit to dual PySide6 host surfaces: a retained single-file developer debug tool and a modular production operator console. §7B reward pipeline remains unchanged.

SECTION 1

System philosophy and operational scope

LSIE-MLF ingests USB-tethered multimodal streams—PCM audio, H.264 video, and chat metadata—and runs concurrent ML pipelines to produce structured metrics. Thompson Sampling uses AU12 smile intensity as a dense proxy reward for the sparse creator-follow event.

The architecture is schema-first and deterministic. USB media transport, temporal synchronization, message passing, and ML inference must behave predictably across independent audio, video, and chat clocks.

The system applies strict data minimization. Raw biometric media does not persist beyond the transient pipeline; persistent storage is limited to derived, anonymized outputs under the Ephemeral Vault policy (§5).

SECTION 2

Deterministic system data flow pipeline

This section defines the complete end-to-end data path through the LSIE-MLF system. Every transition specifies the transport mechanism, payload format, synchronization mechanism, and error behavior.

STAGE 1 · MOBILE HARDWARE → CAPTURE CONTAINER

TRANSPORT	USB passthrough via `/dev/bus/usb` device mapping into the Capture Container; scrcpy v3.3.4 runs as two coordinated instances, with an audio process (`--no-video` plus `--record` to the audio IPC Pipe) on port range 27100:27199 and a video process (`--no-audio` plus `--record` to the video IPC Pipe) on port range 27200:27299.
PAYLOAD	Audio: raw PCM signed 16-bit little-endian (s16le), 48 kHz mono written to `/tmp/ipc/audio_stream.raw`. Video: H.264 carried in an MKV streaming container written to `/tmp/ipc/video_stream.mkv`.
SYNC	Android hardware clock polled via `adb shell` 'echo `$EPOCHREALTIME`' every 30 seconds; drift offset applied to timestamps.
ERROR BEHAVIOR	If USB connection is lost, the Capture Container logs the event, halts pipe writes, and polls reconnection every 2 seconds for 60 seconds before graceful termination.

STAGE 2 · CAPTURE CONTAINER → ORCHESTRATOR CONTAINER

TRANSPORT	Audio IPC Pipe at `/tmp/ipc/audio_stream.raw` via the shared Docker volume ipc-share, consumed by the Orchestrator Container's FFmpeg AudioResampler subprocess. In parallel, the video IPC Pipe `/tmp/ipc/video_stream.mkv` is consumed by PyAV through VideoCapture in the same Orchestrator Container.
PAYLOAD	Continuous byte stream of PCM s16le audio at 48 kHz.
SYNC	The audio FIFO is held open with `exec 3<> /tmp/ipc/audio_stream.raw`, while the video FIFO remains unshielded so a broken PyAV reader forces a fresh MKV header on restart.
ERROR BEHAVIOR	If the Orchestrator Container audio reader crashes, pipe writes return EAGAIN once the buffer fills. Overflowed bytes are silently discarded until the consumer restarts.

STAGE 3 · ORCHESTRATOR CONTAINER INTERNAL: AUDIO RESAMPLING

TRANSPORT	AudioResampler launches the `FFMPEG_RESAMPLE_CMD` subprocess inside the Orchestrator Container to convert `/tmp/ipc/audio_stream.raw` from 48 kHz mono s16le into 16 kHz mono s16le on `pipe:1`.
PAYLOAD	Input: PCM s16le 48 kHz. Output: PCM s16le 16 kHz via `pipe:1`.
SYNC	FFmpeg executes synchronously with the audio processing thread ensuring ordered delivery.
ERROR BEHAVIOR	If FFmpeg exits unexpectedly, the orchestrator detects the failure and restarts the AudioResampler subprocess within 1 second.

STAGE 4 · MODULE B → MODULE C

TRANSPORT	Event dictionaries appended to the orchestrator deque buffer for inclusion in the next segment window.
PAYLOAD	Python dictionaries containing `uniqueId`, `event_type`, `timestamp_utc`, and payload fields.
SYNC	Events timestamped using host UTC and drift-corrected offset before entering the deque ring buffer.
ERROR BEHAVIOR	If the WebSocket disconnects, exponential backoff reconnection is attempted (1s initial delay, 30s max, 10 retries).

STAGE 5 · ORCHESTRATOR CONTAINER → ML WORKER

TRANSPORT	Celery asynchronous task dispatch via `process_segment.delay(payload)` from the Orchestrator Container, through the Redis Message Broker, to the ML Worker.
PAYLOAD	InferenceHandoffPayload JSON object validated against the Pydantic schema.
SYNC	The Orchestrator Container assembles fixed 30-second segments, validates the payload, and dispatches it asynchronously before inference begins in the ML Worker.
ERROR BEHAVIOR	If schema validation fails, the segment is logged and discarded while processing continues.

STAGE 6 · MODULE D → MODULE E

TRANSPORT	Celery asynchronous task dispatch through the Message Broker.
PAYLOAD	JSON payload containing `session_id`, `segment_id`, AU12 intensity, transcription, semantic match, pitch, jitter, and shimmer.
SYNC	Redis append-only persistence ensures tasks survive broker restarts.
ERROR BEHAVIOR	If the broker is unreachable Celery retries with exponential backoff up to five attempts before writing failed payloads to `/data/processed/failed_tasks/`.

STAGE 7 · MODULE E → PERSISTENT STORE

TRANSPORT	psycopg2-binary PostgreSQL connection pool.
PAYLOAD	`SQL INSERT` statements with parameterized queries storing metrics as `DOUBLE PRECISION` and timestamps as `TIMESTAMPTZ`.
SYNC	SERIALIZABLE isolation for experiment updates and READ COMMITTED for metric inserts.
ERROR BEHAVIOR	If the database is unreachable the module buffers up to 1000 records in memory and retries every 5 seconds before overflowing to CSV.

STAGE 8 · API SERVER → ORCHESTRATOR CONTAINER

TRANSPORT	Normalized physiological chunk events are pushed to the Redis list `physio:events` with `RPUSH` by the API Server hydration layer and drained with non-blocking `LPOP` by the Orchestrator Container. The drain is bounded to 100 events per cycle so physiological traffic cannot stall the 30-second segment dispatch cadence.
PAYLOAD	JSON dictionaries matching PhysiologicalChunkEvent, containing `unique_id`, `event_type`, `provider`, `subject_role`, `source_kind`, `window_start_utc`, `window_end_utc`, `ingest_timestamp_utc`, and nested payload fields `ibi_ms_items`, `rmssd_items_ms`, `heart_rate_items_bpm`, `motion_items`, `sample_interval_s`, `valid_sample_count`, `expected_sample_count`, and `derivation_method`.
SYNC	Provider timestamps remain in UTC and freshness is computed at segment assembly time from wall-clock age relative to `source_timestamp_utc` derived from chunk `window_end_utc`. The ADB DriftCorrector is not applied because physiological events do not originate from the Android device clock.
ERROR BEHAVIOR	If signature verification or schema validation fails, the API Server rejects the webhook deterministically (401 or 422). If Redis is unavailable at webhook-ingress time it returns 503 so the provider retries. Hydration failures after successful enqueue are retried asynchronously. Stale samples are marked `is_stale = true` without blocking segment dispatch.

SECTION 3

Codebase architecture and directory mapping

The repository uses a segmented monorepo: each computational domain is isolated by directory but integrated through a root Docker Compose topology. Shared utilities, math libraries, and interface contracts are available without private package registries.

API logic and ML logic are physically separated because the API Server runs an asynchronous FastAPI/Uvicorn event loop while the ML Worker runs GPU-bound Celery tasks. This preserves non-blocking behavior and improves Docker build caching.

3.1 Directory hierarchy

PATH	PURPOSE
`/docker-compose.yml`	The root orchestrator defining the multi-container topology, virtual networks, GPU device reservations, and shared IPC volume mounts. This file is the single entry point for all container lifecycle operations.
`/services/api/`	Houses the API Server logic. Contains the ASGI entry point (Uvicorn), RESTful routing controllers, and dependency injection modules for the Persistent Store connection pool.
`/services/worker/`	Contains the ML Worker Celery task consumers. Houses the orchestration logic that coordinates discrete ML pipeline steps, pulling GPU-bound inference jobs from the Message Broker.
`/services/stream_ingest/`	Dedicated to the Capture Container's raw audio-visual extraction mechanisms. Contains the shell orchestrators responsible for instantiating the IPC Pipe and executing the ADB screen mirroring and audio extraction protocols.
`/packages/ml_core/`	Houses the shared mathematical and inference utilities utilized across the stack. This includes the Python reference implementations for topological facial landmark extraction, Action Unit mathematics, and audio transcription wrappers.
`/packages/schemas/`	Centralizes Pydantic models and JSON Schema Draft 07 definitions ensuring strict interface contracts and data validation rules across services.
`/data/raw/`	Local host volume mount for transient staging of extracted audio-visual buffers including IPC pipe buffers and raw PCM streams. Mounted as tmpfs in production.
`/data/interim/`	Storage for intermediate processing states such as downsampled audio buffers and cropped video frames for localized neural network analysis.
`/data/processed/`	Destination for validated ML inference outputs and structured analytical JSON payloads prior to database ingestion.
`/requirements/`	Maintains deterministic dependency tracking separating base requirements, API dependencies, and heavy ML dependencies for optimized Docker layer caching.
`/data/sql/`	PostgreSQL DDL and seed data files. Mounted read-only to `docker-entrypoint-initdb.d/` for automatic schema initialization on first container startup. Contains experiment definition inserts and metrics table schemas.
`/services/worker/pipeline/`	Module C orchestration and Module E analytics pipeline. Houses the orchestrator, reward computation, and video capture pipeline modules used by the orchestrator and worker containers.

3.2 Build context rules

All Dockerfiles under `/services` MUST use the monorepo root as build context so shared `/packages` code is available during build. `.dockerignore` rules MUST keep unrelated assets out of images. The API Server image excludes heavy ML dependencies; worker images exclude unnecessary web assets.

SECTION 4

Core architecture modules

The modules defined below form the operational spine of the LSIE-MLF specification. Each module contains explicit execution details followed by a formal contract defining inputs, outputs, dependencies, side effects, and failure modes.

4.A Hardware & transport

Purpose. Extract uncompressed, zero-latency audio and video streams from tethered mobile hardware and transport them into the containerized environment without network serialization overhead.

Technology stack. scrcpy v3.3.4, dual-instance scrcpy architecture (audio on port range 27100:27199 and video on port range 27200:27299), IPC Pipe (POSIX Named Pipe) transport for separate audio and video streams, Bash file descriptor manipulation (fd 3 audio shield), Docker shared volume ipc-share

4.A.1 EXECUTION DETAILS

Module A maps the host `/dev/bus/usb` subsystem into the Capture Container and starts two coordinated scrcpy v3.3.4 processes against the same Android device. One process captures audio and the other captures video.

The dual-instance design keeps the transport path simple and restartable. Audio is written directly to `/tmp/ipc/audio_stream.raw` through scrcpy's `--record` path, while video is written directly to `/tmp/ipc/video_stream.mkv`. Splitting the streams avoids cross-stream coupling and allows either scrcpy process to be relaunched independently when only one media path fails.

The capture design uses intentionally asymmetric pipe lifecycle handling so audio continuity and video container recovery each fail in the intended way under independent scrcpy restarts.

Startup is staggered by 4 seconds to prevent concurrent ADB server pushes to the device. Before every capture cycle, the container clears `adb forward` and `adb reverse` tunnels, then launches the audio instance first and the video instance second. Both instances use `--no-playback` and write into the shared ipc-share volume instead of routing media through REST endpoints or Redis payloads.

- Audio scrcpy flags: `--no-video`, `--no-playback`, `--audio-codec=raw`, `--audio-buffer=30`, `--audio-dup`, `--record=$AUDIO_PIPE`, `--record-format=wav`, `--port=27100:27199`, `--tunnel-host=$HOST_IP`.
- Video scrcpy flags: `--no-audio`, `--no-playback`, `--video-codec=h264`, `--max-fps=30`, `--record=$VIDEO_PIPE`, `--record-format=mkv`, `--port=27200:27299`, `--tunnel-host=$HOST_IP`.

> **VIDEO FIFO INTENTIONALLY LEFT UNSHIELDED**
>
> Do not symmetrize the pipe handling. The shell keeps only the audio FIFO open with `exec 3<> $AUDIO_PIPE`; the video FIFO stays unshielded so a PyAV-side failure breaks the write path, forces video scrcpy to exit, and lets the next launch emit a fresh MKV header.

IPC PIPE LIFECYCLE SPECIFICATION

1. **Directory creation:** The Capture Container initializes the shared IPC mount by running `mkdir -p /tmp/ipc` before any scrcpy process is started.
2. **Pipe cleanup and creation:** `setup_pipes()` removes stale FIFO files with `rm -f` and recreates both `/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv` with `mkfifo`.
3. **Audio fd 3 shield:** The shell opens `exec 3<> $AUDIO_PIPE` so the audio pipe stays open across reader or scrcpy restarts and the writer is decoupled from consumer timing.
4. **Video pipe left unshielded:** No extra descriptor is opened for `/tmp/ipc/video_stream.mkv`; if the PyAV consumer drops the pipe, the video scrcpy process exits and the next launch emits a fresh MKV header.
5. **Device wait gate:** `wait_for_device()` polls `adb devices` every 2 seconds for up to 60 seconds and terminates the container if no Android device reports a ready device state.
6. **ADB tunnel cleanup:** Each capture loop starts by running `adb forward --remove-all` and `adb reverse --remove-all` so stale tunnels cannot collide with the audio and video port ranges.
7. **Audio stream launch:** The audio scrcpy instance launches first and writes directly to `/tmp/ipc/audio_stream.raw` with `--record` while binding port range 27100:27199 through `$HOST_IP`.
8. **Staggered video launch:** After a 4-second delay, the video scrcpy instance launches and writes H.264 in an MKV streaming container directly to `/tmp/ipc/video_stream.mkv` on port range 27200:27299.
9. **Monitoring and restart:** The container waits on both scrcpy PIDs with `wait -n`; when either process exits, both children are cleaned up, the device is rechecked, and the dual-stream capture loop restarts.
10. **Cross-launch cleanup:** On the next container start, `setup_pipes()` again runs `rm -f` on both FIFO paths before recreating them, preventing stale shared-volume pipe files from surviving across launches.

INPUTS	USB-connected Android device producing raw PCM audio (s16le 48 kHz) and H.264 video via scrcpy.
OUTPUTS	Continuous PCM audio stream (s16le, 48 kHz, mono) written to `/tmp/ipc/audio_stream.raw`, and continuous H.264 video in an MKV streaming container written to `/tmp/ipc/video_stream.mkv`; both are exposed as IPC Pipes on the shared Docker volume ipc-share.
DEPENDENCIES	scrcpy v3.3.4, ADB server via `ADB_SERVER_SOCKET=tcp:host.docker.internal:5037`, `/dev/bus/usb` device passthrough, Docker volume ipc-share mounted at `/tmp/ipc/`, environment variables `SDL_VIDEODRIVER=dummy`, `XDG_RUNTIME_DIR=/tmp`, `SCRCPY_SERVER_PATH`, and `SCRCPY_ICON_PATH`, plus dedicated scrcpy port ranges 27100:27199 (audio) and 27200:27299 (video).
SIDE EFFECTS	Creates two named pipes (`/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv`), opens file descriptor 3 as an audio-pipe shield, spawns persistent audio and video scrcpy processes, and establishes ADB forward/reverse tunnels for port ranges 27100:27199 and 27200:27299.
FAILURE MODES	USB disconnect causes `wait_for_device()` to poll every 2 seconds for up to 60 seconds before the container exits. If the audio scrcpy process crashes, `wait -n` detects the exit, both scrcpy PIDs are cleaned up, and the loop restarts while fd 3 keeps `/tmp/ipc/audio_stream.raw` open. If the video path breaks, the unshielded `/tmp/ipc/video_stream.mkv` pipe lets the video scrcpy process fail so the next launch writes a fresh MKV header for PyAV. Each capture cycle removes adb forward/reverse tunnels to prevent port collisions, and if the downstream consumer lags, pipe writes may return EAGAIN so overflowed data is silently discarded per §12.

4.B Ground truth ingestion

Purpose. Ingest real-time contextual metadata, chat streams, and user interactions to create ground-truth datasets.

Technology stack. Python 3.11.x, TikTokLive connector, EulerStream signature API

4.B.1 EXECUTION DETAILS

The system continuously ingests live chat and interaction events from the platform using the TikTokLive Python connector.

Connection requires ByteDance anti-crawler signatures including `X-Bogus` and `msToken`. Signature generation is delegated to the EulerStream signature server.

Once connected the ingestion module enforces the `Action_Combo` constraint where multiple events must coincide temporally to form a valid ground truth signal.

Incoming protobuf messages are parsed into Python dictionaries and high-value composite events are staged for later synchronization with inference outputs.

4.B.2 PHYSIOLOGICAL INGESTION ADAPTER

Physiological webhook ingestion is implemented as a FastAPI POST route in the API Server container. It is separate from TikTok WebSocket ingestion and has its own authentication, validation, and transport rules.

Oura webhook deliveries trigger an OAuth2-backed hydration step that fetches the updated provider resource and normalizes it into physiological chunk records.

- Verify `x-oura-signature` with `OURA_WEBHOOK_SECRET`, resolve `subject_role`, and suppress duplicates with Redis `SETNX physio:seen:{unique_id}`.
- Parse only the notification metadata needed to route hydration.
- Enqueue hydration work on `physio:hydrate`; the hydrator refreshes tokens as needed and emits normalized PhysiologicalChunkEvent records using `source_kind` ibi or session.
- Append normalized chunk events to `physio:events` with `RPUSH` for Orchestrator consumption.
- The route emits explicit HTTP failures for signature, schema, and Redis-availability problems.

OURA WEBHOOKS ARE CHANGE NOTIFICATIONS, NOT RMSSD SAMPLES

Treat Oura Ring webhooks as change notifications only. Do not read them as authoritative `rmssd_ms` samples or derive RMSSD from generic heartrate notifications; verify, deduplicate, hydrate the provider resource, and normalize the result into PhysiologicalChunkEvent records.

RAW WEBHOOK/PROVIDER PAYLOADS MUST NEVER PERSIST

Keep raw webhook JSON and hydrated provider response bodies transient. They MUST NOT be persisted; only normalized downstream records may continue, with `401` on signature failure, `422` on schema failure, and `503` when Redis is unavailable.

INPUTS	Authenticated TikTok Webcast WebSocket event stream using EulerStream-signed URL parameters. Additionally, authenticated physiological webhook payloads arrive via the API Server and are enqueued to a Redis list for downstream consumption (§4.B.2).
OUTPUTS	Parsed dictionaries representing high-value events containing `uniqueId`, `event_type`, `timestamp_utc`, `gift_value`, and `combo_events`.
DEPENDENCIES	TikTokLive library, EulerStream signature API, network access to TikTok Webcast servers.
SIDE EFFECTS	Maintains persistent WebSocket connection and refreshes signature tokens during reconnects.
FAILURE MODES	WebSocket disconnection triggers exponential backoff reconnect. EulerStream outage results in degraded mode. Malformed protobuf events are logged and discarded.

4.C Orchestration & synchronization

Purpose. Module C runs as a dedicated Orchestrator Container built from the worker image and started with `python3.11 -m services.worker.run_orchestrator`. It polls ADB for drift correction, resamples 48 kHz PCM to 16 kHz through a persistent FFmpeg subprocess, processes video frames for AU12, assembles fixed 30-second segments, and dispatches InferenceHandoffPayload tasks to the Celery worker.

Technology stack. asyncio event loop, FFmpeg persistent resampling subprocess, PyAV-backed video pipe consumer, MediaPipe Face Mesh via packages/ml_core, ADB epoch polling, Celery task dispatch, Redis broker, psycopg2/PostgreSQL session registration and experiment lookup, collections.deque buffering

4.C.1 TEMPORAL DRIFT CORRECTION

DriftCorrector polls the Android epoch every 30 seconds by invoking ADB and computes the drift offset as host UTC minus Android epoch before segment timestamps are emitted.

Each corrected timestamp is produced by adding the cached drift offset to the original timestamp, and the orchestrator warns when absolute drift exceeds 150 milliseconds.

When ADB polling degrades, drift correction falls back to deterministic cached-offset behavior instead of introducing unstable timestamp jitter.

> **DRIFT CORRECTION FREEZES AFTER REPEATED ADB FAILURES**
>
> After three consecutive ADB poll failures, DriftCorrector freezes the last cached offset and keeps returning it while frozen. If the frozen state lasts 300 seconds, reset the offset to zero until a later successful poll refreshes it.

4.C.2 VIDEO TELEMETRY AND SEGMENT ASSEMBLY

The Orchestrator Container keeps a long-lived video capture thread on `/tmp/ipc/video_stream.mkv`, samples the latest frame inside the asyncio loop, and lazy-loads MediaPipe Face Mesh together with the AU12 normalizer on first use.

Before stimulus injection, incoming frames only accumulate the *B_neutral* calibration baseline. After the greeting trigger is recorded, the orchestrator stores a drift-corrected stimulus timestamp and appends per-frame AU12 observations `{timestamp_s, intensity}` to `_au12_series` for downstream reward computation.

In parallel, audio bytes accumulate until a full 30-second window is available. The orchestrator then drains the event deque, assembles an InferenceHandoffPayload, attaches `_active_arm`, `_experiment_id`, `_expected_greeting`, `_stimulus_time`, `_au12_series`, and `_x_max`, base64-encodes binary fields for Celery JSON transport, and dispatches the segment to Module D.

4.C.3 AUDIO RESAMPLING

AudioResampler launches a persistent FFmpeg subprocess inside the Orchestrator Container to convert the raw 48 kHz mono IPC stream at `/tmp/ipc/audio_stream.raw` into 16 kHz mono s16le PCM emitted on `pipe:1`.

The read loop runs at video cadence so audio consumption stays synchronized with 30 FPS segment assembly.

If the FFmpeg subprocess is not running, `read_chunk()` waits 1 second and starts a fresh resampler. EOF or pipe read errors stop the old process so the next iteration can recover automatically.

> **AUDIO CHUNKING IS 1/30 SECOND, NOT 1 SECOND**
>
> Consume audio in 1/30-second reads. With `bytes_per_second` = 32000, set `chunk_size` = `int(bytes_per_second / 30)` so each iteration reads 1,066 bytes (approximately 1,067), not a 1-second 32000-byte chunk.

4.C.4 PHYSIOLOGICAL STATE BUFFER

Module C keeps an in-memory Physiological State Buffer so 30-second inference segments can use wearable data that arrives asynchronously and at irregular cadence.

The Orchestrator drains chunk events from Redis, updates a rolling per-subject buffer, derives a current scalar snapshot over the trailing derivation window, and may attach the resulting physiological context to the outgoing segment payload.

- Store a time-ordered deque per `subject_role` covering at least `PHYSIO_BUFFER_RETENTION_S` seconds (default 900). Drain at most 100 events per cycle from `physio:events`.
- Derive each snapshot over `PHYSIO_DERIVE_WINDOW_S` seconds (default 300). Prefer overlapping ibi chunks; otherwise use overlapping session chunks; otherwise `rmssd_ms = null`.

- IBI-derived RMSSD uses `RMSSD = sqrt((1 / (N - 1)) * sum_{i=1}^{N-1}(IBI_{i+1} - IBI_i)^2)`. Heart rate is the rounded mean of overlapping `heart_rate_items_bpm` values or null.
- Compute `validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` and set `is_valid = validity_ratio ⩾ PHYSIO_VALIDITY_MIN` (default 0.80). Invalid windows force `rmssd_ms = null`.
- Set `source_timestamp_utc` from the latest contributing `window_end_utc`, carry `source_kind` and `derivation_method`, and compute `freshness_s` from wall clock time without using ADB drift correction.
- Use `PHYSIO_STALENESS_THRESHOLD_S` (default 600) to classify snapshot age and surface staleness metadata on derived snapshots.

_PHYSIOLOGICAL_CONTEXT IS OPTIONAL

Omit `_physiological_context` entirely when no physiological snapshot exists. Do not emit empty objects, null-filled placeholders, or fabricated defaults.

STALE PHYSIOLOGY DOES NOT BLOCK DISPATCH

When `freshness_s` exceeds `PHYSIO_STALENESS_THRESHOLD_S`, mark `is_stale = True` and continue dispatch. Staleness annotates the payload; it does not block segment assembly or downstream inference.

DRIFT POLLING SPECIFICATION

PARAMETER	VALUE
Polling interval	30 seconds (DRIFT_POLL_INTERVAL)
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch string with microsecond precision parsed as float
Maximum tolerated drift	150 milliseconds (MAX_TOLERATED_DRIFT_MS)
Drift formula	drift_offset = host_utc - android_epoch
Timestamp correction	corrected_ts = original_ts + drift_offset
Freeze threshold	3 consecutive ADB failures (DRIFT_FREEZE_AFTER_FAILURES)
Reset timeout	300 seconds / 5 minutes of frozen state (DRIFT_RESET_TIMEOUT)

FFMPEG RESAMPLING COMMAND

FFmpeg resampling command — BASH

```bash
ffmpeg -f s16le -ar 48000 -ac 1 -i /tmp/ipc/audio_stream.raw -ar 16000 -f s16le -ac 1 pipe:1
```

INPUTS	Raw PCM audio from `/tmp/ipc/audio_stream.raw`, H.264 video frames decoded from `/tmp/ipc/video_stream.mkv` by the PyAV-backed video capture thread, ground-truth event dictionaries drained from the in-memory deque, Android epoch timestamps returned by ADB drift polls, and operator stimulus trigger events used to mark greeting injection time.
OUTPUTS	16 kHz resampled mono PCM, drift-corrected segment timestamps, InferenceHandoffPayload segments dispatched as Celery tasks to the ML Worker, per-frame AU12 telemetry in `_au12_series`, stimulus timestamps in `_stimulus_time`, and experiment attribution fields `_active_arm`, `_experiment_id`, `_expected_greeting`, and `_x_max`.
DEPENDENCIES	FFmpeg persistent subprocess, PyAV-backed video capture, MediaPipe Face Mesh and AU12 normalization via `packages/ml_core`, ADB drift polling, Celery task dispatch, Redis broker, PostgreSQL access for session registration and Thompson Sampling arm selection, shared Docker volume ipc-share mounted at `/tmp/ipc/`, and data-raw, data-interim, and data-processed volumes.
SIDE EFFECTS	Spawns a persistent FFmpeg subprocess, starts and revives the video capture thread, registers the session in PostgreSQL, selects a Thompson Sampling arm and expected greeting, accumulates AU12 calibration and telemetry state in memory, and dispatches assembled segments to the Celery worker.
FAILURE MODES	ADB poll failures freeze the last drift offset after 3 consecutive errors and reset it to zero after 300 seconds of frozen state. Missing or crashed FFmpeg subprocesses wait 1 second before automatic restart, while EOF and read errors stop the old process so the next iteration can recover. Video capture thread death triggers a best-effort stop/recreate/start cycle in the run loop after pipe crashes, AU12 frame processing failures are debug-logged and skipped, event buffer overflow is bounded by `deque(maxlen=10000)` eviction, and Celery dispatch failures are logged while the loop continues.

4.D Multimodal ML processing

Purpose. Perform speech transcription, facial landmark inference, acoustic analysis, and semantic evaluation.

Technology stack. faster-whisper v1.2.1, mediapipe v0.10.9, praat-parselmouth v0.4.4, spacy v3.7.x

4.D.1 SPEECH TRANSCRIPTION

Speech transcription is executed by TranscriptionEngine, which wraps faster-whisper with the CTranslate2 backend on the pinned worker runtime.

For each segment, the Celery task stages the 16 kHz s16le PCM bytes through a temporary file and uses FFmpeg to prepare a transcribable audio container before calling `engine.transcribe()`.

GPU execution targets CUDA 12 with cuDNN 8, matching the worker image. cuDNN 8 is required to preserve Pascal SM 6.1 dp4a compatibility for INT8 inference on the target hardware (GTX 1080 Ti).

COMPUTE_TYPE IS FIXED TO INT8

`compute_type` is not a deployment-time knob. TranscriptionEngine fixes `_COMPUTE_TYPE` to `"int8"` to match the Pascal / cuDNN 8 inference path and prevent accidental misconfiguration.

WRAP RAW PCM IN WAV BEFORE TRANSCRIPTION

Before calling `engine.transcribe()`, wrap the temporary 16 kHz s16le PCM bytes in a WAV container with FFmpeg. Headerless PCM is not the intended direct ASR input.

4.D.2 COMPUTER VISION — FACE MESH

Video frames are processed using MediaPipe Face Mesh generating a 478-vertex 3D landmark mesh.

The model uses Procrustes Analysis to construct a normalized metric face space.

4.D.3 ACOUSTIC ANALYSIS

The Praat engine accessed through parselmouth extracts pitch, jitter, and shimmer metrics from speech audio.

4.D.4 TEXT PRE-PROCESSING

spaCy performs tokenization, normalization, and linguistic preprocessing prior to LLM semantic evaluation.

MODULE D — FORMAL CONTRACT § 4.D.contract

INPUTS	16 kHz audio stream, raw video frames, and InferenceHandoffPayload segments.
OUTPUTS	Speech transcripts, 478-vertex facial landmark arrays, AU12 intensity scores, acoustic metrics, semantic evaluation JSON results, and RewardResult payloads produced by invoking `compute_reward()` (§7B) with the segment's `_au12_series`, `_stimulus_time`, and semantic evaluation `is_match` gate.
DEPENDENCIES	faster-whisper 1.2.1 with CTranslate2, MediaPipe 0.10.9, praat-parselmouth 0.4.4, spaCy 3.7.*, OpenAI Python SDK for Azure OpenAI calls, FFmpeg for WAV container wrapping, and CUDA 12.2.2 runtime with cuDNN 8.
SIDE EFFECTS	Loads ML models into GPU memory and performs external HTTPS calls to Azure OpenAI.
FAILURE MODES	GPU OOM triggers worker restart. Whisper load failure aborts startup. Missing face returns null facial metrics. LLM timeout retries once before recording null.

4.E Experimentation & analytics

Purpose. Module E aggregates inference outputs into persistent analytics, computes rewards, updates Thompson Sampling state, and serves operator-facing analytics through FastAPI-backed PySide6 surfaces.

Technology stack. psycopg2-binary v2.9.x, pandas v2.2.x, SciPy, FastAPI analytics APIs, PySide6 operator surfaces (host)

4.E.1 EXECUTION DETAILS

Module E aggregates high-frequency inference outputs, reward analytics, and physiological/co-modulation metrics into structured time-series records.

Reward computation continues to follow §7B and returns a RewardResult dataclass from the established reward path.

Operator visualization now runs on the operator host through two PySide6 surfaces, while all data access continues to flow through the FastAPI server.

- `scripts/debug_studio.py`: single-file developer diagnostic GUI for live video, landmarks, AU12 normalization, and analytics polling.
- `services/operator_console/`: modular production console for session monitoring, experiment readback, physiology/co-modulation views, and operator actions.

> **REWARD ENTRY POINT REMAINS COMPUTE_REWARD()**
>
> Physiological extensions do not change §7B reward computation. `compute_reward()` remains the entry point and still consumes `_au12_series`, `_stimulus_time`, and `is_match`.

4.E.2 PHYSIOLOGICAL CO-MODULATION ANALYTICS

Module E computes rolling physiological co-modulation between the operator and the streamer using valid RMSSD snapshots derived in Module C.

A positive Pearson correlation indicates autonomic alignment and a negative value indicates divergence; the metric is stored as analytics output alongside other session measurements.

- Persist one physiological snapshot per subject per segment to `physiology_log`, including RMSSD, heart rate, validity, freshness, staleness, provider, `source_kind`, and `derivation_method`.
- Resample eligible RMSSD snapshots to 1-minute bins and compute Pearson correlation over a 10-minute rolling window.
- Coverage and variance checks determine whether a Pearson value can be emitted for a window.

- Persist `session_id`, window metadata, Pearson `r`, pair count, coverage ratio, and per-subject RMSSD means to `comodulation_log`.

CO-MODULATION IS OBSERVATIONAL ONLY

Treat the Co-Modulation Index as logging and analytics output only. It MUST NOT feed `compute_reward()`, semantic gating, Thompson Sampling updates, or any other reward logic.

NULL CO-MODULATION IS AN EXPECTED RESULT

A null co-modulation value is valid when there are fewer than 4 paired observations, zero variance in either aligned series, or insufficient eligible overlap. Record the null and continue; do not treat it as a pipeline error.

MODULE E — FORMAL CONTRACT **§ 4.E.contract**

INPUTS	Inference payloads from Module D delivered through the Celery task queue.
OUTPUTS	Time-series records stored in the Persistent Store, Thompson Sampling experiment assignments and posterior updates (`α += r_t`, `β += 1 - r_t`), RewardResult dataclass produced by `compute_reward()` (§7B), and operator-facing analytics and session-state read APIs consumed by PySide6 host surfaces.
DEPENDENCIES	psycopg2-binary, pandas, SciPy, Redis Message Broker, PostgreSQL database.
SIDE EFFECTS	Writes metrics and reward results to database tables, updates Thompson Sampling Beta posterior parameters per encounter, and exposes analytics and experiment state through the FastAPI server for consumption by PySide6 operator surfaces.
FAILURE MODES	Database outage buffers 1000 records before disk overflow. Celery task deserialization errors are logged and discarded.

4.F Context enrichment

Purpose. Perform asynchronous scraping of external web sources to enrich contextual information about the stream.

Technology stack. Celery v5.4.x, Message Broker, patchright v1.58.2

4.F.1 EXECUTION DETAILS

Context enrichment tasks run asynchronously outside the main inference loops.

Celery workers execute scraping jobs that gather external metadata about creators or trending topics.

patchright is used instead of standard Playwright to bypass bot-detection mechanisms by patching Chromium automation fingerprints.

INPUTS	Celery tasks containing target URL, `scrape_type`, and `timeout_seconds` parameters.
OUTPUTS	JSON objects containing scraped metadata including `source_url`, `scraped_at_utc` timestamp, and extracted data dictionary.
DEPENDENCIES	Celery, Redis Message Broker, patchright browser automation toolkit, Chromium runtime.
SIDE EFFECTS	Launches ephemeral headless browser instances and performs outbound HTTP requests.
FAILURE MODES	CAPTCHA or HTTP blocking returns empty result. Browser crash triggers retry. Timeout terminates browser. Broker outage triggers Celery reconnection.

SECTION 5

Data governance and security

LSIE-MLF processes sensitive speech and biometric data. The system therefore enforces strict data minimization, encryption, and retention controls so personally identifiable media does not persist beyond transient processing.

5.1 Ephemeral vault specification

PARAMETER	SPECIFICATION
Encryption algorithm	AES-256-GCM (authenticated encryption with associated data)
Encryption library	PyCryptodome (pycryptodome >= 3.20.0)
Key length	256 bits
IV/Nonce length	96 bits (12 bytes) generated using `os.urandom(12)`
Key generation	`os.urandom(32)` generating a unique key per debug session
Key storage location	Process memory only. Keys are never written to disk, environment variables, or configuration files.
Key lifecycle	Generated at session start and destroyed when the container terminates. No persistence mechanism exists.
Secure deletion method	`shred -vfz -n 3` executed by internal cron every 24 hours on `/data/raw/` and `/data/interim/`
Retention period	Maximum 24 hours for any raw media buffer touching physical disk

5.2 Data classification

Transient Data

Raw PCM audio buffers and uncompressed video frames existing only in volatile memory during processing. This tier also includes raw Oura Ring webhook JSON payloads and normalized PhysiologicalChunkEvent records while they remain in volatile queue transit or other transient memory-backed paths.

STORAGE	/tmp/ipc/ (tmpfs) and kernel pipe buffers and Redis list `physio:events`
RETENTION	Duration of active processing only with no disk persistence. Physiological webhook payloads MUST remain transient; if they ever touch disk for debugging, they inherit Ephemeral Vault controls and the 24-hour secure deletion cycle.

Debug Storage

Encrypted diagnostic media buffers written to disk only when algorithmic anomalies occur.

STORAGE	/data/raw/ and /data/interim/
RETENTION	Maximum 24 hours followed by secure deletion.

Permanent Analytical Storage

Anonymized analytical metrics including AU12 scores, acoustic metrics, and semantic evaluation results.

STORAGE	Persistent Store (PostgreSQL)
RETENTION	Indefinite subject to database backup policies.

The system MUST NOT store raw facial images, voiceprints, raw Oura webhook JSON, or raw PhysiologicalChunkEvent payloads in the Persistent Store. Persistent storage is limited to derived, normalized outputs such as AU12 metrics, scalar RMSSD, heart rate, validity metadata, and co-modulation analytics.

SECTION 6

Interface contracts and payload schemas

Distributed multimodal pipelines require strict interface contracts. Incorrect array serialization or missing fields can break downstream indexing, alignment, and persistence.

LSIE-MLF uses JSON Schema Draft 07 and Pydantic validation to keep module boundaries deterministic at runtime.

The schemas below define the base media transport contract and the experiment attribution fields that the Orchestrator attaches before Celery dispatch. These fields carry arm selection, stimulus timing, AU12 telemetry, and calibration state for the reward pipeline (§7B).

Two physiological schemas are defined here. PhysiologicalChunkEvent (§6.2) describes the normalized chunk emitted by the API Server hydration path and consumed by the Orchestrator. PhysiologicalContext (§6.3) describes the optional `_physiological_context` snapshot attached to InferenceHandoffPayload. The payload field name is `physiological_sample_event_schema`; its caption and schema title refer to PhysiologicalChunkEvent.

6.1 InferenceHandoffPayload — JSON schema draft 07

InferenceHandoffPayload — JSON schema draft 07 **JSON**

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "InferenceHandoffPayload",
 "description": "Standardized schema for multimodal ML pipeline handoff. Base contract defines
 ↪  media transport fields; v3 experiment attribution fields support Thompson Sampling reward
 ↪  computation.",
 "type": "object",
 "required": [
  "session_id",
  "timestamp_utc",
  "media_source",
  "segments"
 ],
 "properties": {
  "session_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$",
   "description": "UUID v4 representing the continuous live stream session."
  },
  "timestamp_utc": {
   "type": "string",
   "format": "date-time",
   "description": "RFC 3339 UTC timestamp of inference event completion."
  },
  "media_source": {
   "type": "object",
   "required": [
    "stream_url",
```

```
      "codec",
      "resolution"
     ],
     "properties": {
      "stream_url": {
       "type": "string",
       "format": "uri"
      },
      "codec": {
       "type": "string",
       "enum": [
        "h264",
        "h265",
        "raw"
       ]
      },
      "resolution": {
       "type": "array",
       "items": {
        "type": "integer",
        "minimum": 1
       },
       "minItems": 2,
       "maxItems": 2,
       "description": "[width, height] in pixels."
      }
     }
    },
    "segments": {
     "type": "array",
     "description": "Array of analyzed video segments."
    },
    "_active_arm": {
     "type": "string",
     "description": "Thompson Sampling arm identifier selected for this segment."
    },
    "_experiment_id": {
     "type": "integer",
     "description": "Persistent Store experiment row ID for this session."
    },
    "_expected_greeting": {
     "type": "string",
     "description": "Greeting rule text assigned to the active arm for semantic evaluation."
    },
    "_stimulus_time": {
     "type": [
      "number",
      "null"
     ],
     "description": "Drift-corrected UTC epoch seconds of stimulus injection, or null if no
     ↪  stimulus occurred in this segment."
    },
    "_au12_series": {
     "type": "array",
     "items": {
      "type": "object",
      "required": [
       "timestamp_s",
       "intensity"
      ],
```

```
      "properties": {
       "timestamp_s": {
        "type": "number"
       },
       "intensity": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 1.0
       }
      }
     },
     "description": "Per-frame AU12 bounded intensity observations from the orchestrator."
    },
    "_x_max": {
     "type": [
      "number",
      "null"
     ],
     "description": "Per-subject maximum raw D_mouth/IOD ratio observed during calibration, used
     ↪  for optional range normalization. Null if calibration is incomplete."
    }
   }
}
```

6.2 PhysiologicalSampleEvent — JSON schema draft 07

PhysiologicalSampleEvent — JSON schema draft 07 **JSON**

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalChunkEvent",
 "type": "object",
 "required": [
  "unique_id",
  "event_type",
  "provider",
  "subject_role",
  "source_kind",
  "window_start_utc",
  "window_end_utc",
  "ingest_timestamp_utc",
  "payload"
 ],
 "properties": {
  "unique_id": {
   "type": "string",
   "format": "uuid"
  },
  "event_type": {
   "type": "string",
   "enum": [
    "physiological_chunk"
   ]
  },
  "provider": {
```

```
      "type": "string",
      "enum": [
        "oura"
      ]
    },
    "subject_role": {
      "type": "string",
      "enum": [
        "streamer",
        "operator"
      ]
    },
    "source_kind": {
      "type": "string",
      "enum": [
        "ibi",
        "session"
      ]
    },
    "window_start_utc": {
      "type": "string",
      "format": "date-time"
    },
    "window_end_utc": {
      "type": "string",
      "format": "date-time"
    },
    "ingest_timestamp_utc": {
      "type": "string",
      "format": "date-time"
    },
    "payload": {
      "type": "object",
      "required": [
        "sample_interval_s",
        "valid_sample_count",
        "expected_sample_count",
        "derivation_method"
      ],
      "properties": {
        "ibi_ms_items": {
          "type": "array",
          "items": {
            "type": "number",
            "exclusiveMinimum": 0.0
          }
        },
        "rmssd_items_ms": {
          "type": "array",
          "items": {
            "type": "number",
            "minimum": 0.0
          }
        },
        "heart_rate_items_bpm": {
          "type": "array",
          "items": {
            "type": "integer",
            "minimum": 20,
            "maximum": 300
```

```
      }
     },
     "motion_items": {
      "type": "array",
      "items": {
       "type": "number",
       "minimum": 0.0
      }
     },
     "sample_interval_s": {
      "type": "integer",
      "exclusiveMinimum": 0
     },
     "valid_sample_count": {
      "type": "integer",
      "minimum": 0
     },
     "expected_sample_count": {
      "type": "integer",
      "minimum": 0
     },
     "derivation_method": {
      "type": "string",
      "enum": [
       "provider",
       "server"
      ]
     }
    },
    "additionalProperties": false,
    "anyOf": [
     {
      "required": [
       "ibi_ms_items"
      ]
     },
     {
      "required": [
       "rmssd_items_ms"
      ]
     },
     {
      "required": [
       "heart_rate_items_bpm"
      ]
     }
    ]
   }
  },
  "additionalProperties": false
 }
```

6.3 PhysiologicalContext — JSON schema draft 07

PhysiologicalContext — JSON schema draft 07 JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalContext",
 "type": "object",
 "properties": {
  "streamer": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  },
  "operator": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  }
 },
 "$defs": {
  "PhysiologicalSnapshot": {
   "type": "object",
   "required": [
    "source_timestamp_utc",
    "freshness_s",
    "is_stale",
    "provider",
    "source_kind",
    "derivation_method",
    "window_s",
    "validity_ratio",
    "is_valid"
   ],
   "properties": {
    "rmssd_ms": {
     "type": [
      "number",
      "null"
     ],
     "minimum": 0.0
    },
    "heart_rate_bpm": {
     "type": [
      "integer",
      "null"
     ],
     "minimum": 20,
     "maximum": 300
```

```
      },
      "source_timestamp_utc": {
       "type": "string",
       "format": "date-time"
      },
      "freshness_s": {
       "type": "number",
       "minimum": 0.0
      },
      "is_stale": {
       "type": "boolean"
      },
      "provider": {
       "type": "string"
      },
      "source_kind": {
       "type": "string",
       "enum": [
        "ibi",
        "session"
       ]
      },
      "derivation_method": {
       "type": "string",
       "enum": [
        "provider",
        "server"
       ]
      },
      "window_s": {
       "type": "integer",
       "exclusiveMinimum": 0
      },
      "validity_ratio": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      },
      "is_valid": {
       "type": "boolean"
      }
     },
     "additionalProperties": false
    }
   }
  }
```

SECTION 7

Mathematical specifications

7A AU12 facial action unit geometry

AU12 measures lip-corner pull driven by the zygomaticus major. In LSIE-MLF it is the primary facial-affect signal used for reward estimation.

Module D computes AU12 intensity from the 478 three-dimensional MediaPipe Face Mesh landmarks. Geometric normalization uses interocular distance (IOD) as the scale reference so the metric remains stable across head distance and pose.

7A.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_61	`landmarks[61]`	Left lip corner landmark.
L_291	`landmarks[291]`	Right lip corner landmark.
L_33	`landmarks[33]`	Left eye outer corner.
L_133	`landmarks[133]`	Left eye inner corner.
L_362	`landmarks[362]`	Right eye inner corner.
L_263	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_133).
E_R	`e_r`	Geometric center of right eye midpoint(L_-362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_mouth	`d_mouth`	3D Euclidean distance between lip corners.
$B_neutral$	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_AU12(t)$	`i_au12`	Normalized AU12 intensity at time t.
$alpha$	`self.alpha`	Empirical linear projection multiplier. Default 6.0; calibrated so a Duchenne deviation of approximately 0.15 maps to approximately 0.90 before tanh bounding.
$Score_AU12(t)$	`score_au12`	Final AU12 FACS score in range 0–5.
$Score_{AU12,bounded}(t)$	`compute_bounded_intensity()`	AU12 intensity bounded to [0.0, 1.0] via tanh soft-saturation. Output method for the fractional Beta-Bernoulli Thompson Sampling reward pipeline.

STEP 1: TOPOLOGICAL LANDMARK EXTRACTION

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

- Mouth corners: L_61 (left) and L_291 (right)

- Left eye bounds: L_33 outer and L_133 inner
- Right eye bounds: L_362 inner and L_263 outer

STEP 2: DERIVATION OF INTEROCULAR DISTANCE (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

STEP 3: AU12 DISTANCE AND BASELINE CALIBRATION

The raw AU12 distance equals the Euclidean distance between lip corners.

A neutral baseline B_neutral is computed from the rolling average of D_mouth/IOD during the calibration phase.

The bounded output maps [0, +infinity) to [0, 1) via tanh soft-saturation, providing a continuous gradient at the boundaries suitable for fractional Beta-Bernoulli posterior updates. With alpha_scale=6.0, a Duchenne smile (deviation approximately 0.15) maps to tanh(0.9) approximately 0.72, and laughter (deviation approximately 0.20) maps to tanh(1.2) approximately 0.83.

$$D_{mouth} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{AU12}(t) = (D_{mouth}(t)/IOD(t)) - B_{neutral}$$

$$Score_{AU12}(t) = max(0, \alpha \cdot I_{AU12}(t))$$

$$Score_{AU12,bounded}(t) = \tanh\big(\alpha \cdot \max(0,\ I_{AU12})\big)$$

7A.2 AU12NORMALIZER — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import math
from typing import Any
import numpy as np
import numpy.typing as npt

EPSILON: float = 1e-6
DEFAULT_ALPHA_SCALE: float = 6.0  # §7A: 0.90 / 0.15 = 6.0
```

```python
class AU12Normalizer:
    def __init__(self, alpha: float = DEFAULT_ALPHA_SCALE) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: list[float] = []

    def _extract_geometry(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> tuple[float, float] | None:
        left_eye_outer = landmarks[33]
        left_eye_inner = landmarks[133]
        right_eye_inner = landmarks[362]
        right_eye_outer = landmarks[263]
        left_lip = landmarks[61]
        right_lip = landmarks[291]

        e_l = (left_eye_outer + left_eye_inner) / 2.0
        e_r = (right_eye_inner + right_eye_outer) / 2.0
        iod: float = float(np.linalg.norm(e_r - e_l))
        if iod < EPSILON:
            return None
        d_mouth: float = float(np.linalg.norm(right_lip - left_lip))
        return (iod, d_mouth)

    def _update_calibration(self, ratio: float) -> None:
        self.calibration_buffer.append(ratio)
        self.b_neutral = float(np.mean(self.calibration_buffer))

    def compute_raw_ratio(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> float | None:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return None
        iod, d_mouth = geom
        return d_mouth / iod

    def compute_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        score = self.alpha * (ratio - self.b_neutral)
        return float(min(max(score, 0.0), 5.0))

    def compute_bounded_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
```

```
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        deviation = max(0.0, ratio - self.b_neutral)
        return float(math.tanh(self.alpha * deviation))
```

7B Thompson sampling reward pipeline

LSIE-MLF uses Thompson Sampling to select greeting strategies that maximize creator engagement. The reward is derived from AU12 smile intensity (§7A), which serves as a dense proxy for the sparse creator-follow event.

The reward pipeline applies a binary semantic gate so only encounters where the greeting was actually delivered contribute to posterior updates. Non-attributable smiles do not update the posterior.

7B.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
t_{stim}	`stimulus_time_s`	Drift-corrected UTC epoch timestamp of greeting stimulus injection.
$W_{measure}$	`extract_stimulus_window()`	Stimulus-locked measurement window [*t_stim* + 0.5s, *t_stim* + 5.0s].
$W_{baseline}$	`extract_stimulus_-window(baseline)`	Pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s].
P_{90}	`p90_intensity`	90th percentile of bounded AU12 intensities within *W_measure*.
G_t	`semantic_gate`	Binary semantic validity gate: 1 if `is_match` is True, 0 otherwise.
r_t	`gated_reward`	Gated reward signal: *r_t* = *P_90* × *G_t*. Range [0.0, 1.0].
α	`posterior_alpha`	Beta distribution alpha parameter (success count). Prior: 1.0.
β	`posterior_beta`	Beta distribution beta parameter (failure count). Prior: 1.0.
x_{max}	`_x_max`	Per-subject maximum raw *D_mouth*/IOD ratio from calibration, used for optional range normalization.
$B_{neutral,pre}$	`baseline_b_neutral`	Mean AU12 bounded intensity in *W_baseline*, or null when no pre-stimulus frames exist.

STEP 1: SIGNAL SOURCE AND BOUNDED OUTPUT

The reward pipeline consumes per-frame bounded AU12 intensities from `compute_bounded_intensity()` (§7A). Each observation is a `{timestamp_s, intensity}` pair with intensity in [0.0, 1.0].

STEP 2: STIMULUS WINDOW EXTRACTION

The measurement window is defined as [*t_stim* + 0.5s, *t_stim* + 5.0s]. The 0.5-second onset delay accounts for the physiological latency between hearing the greeting and producing a facial response. The 5.0-second end bound captures the full smile trajectory including onset, apex, and initial decay.

A pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s] is computed for diagnostic purposes. The 2.0-second guard prevents contamination from anticipatory expressions triggered by the operator's preparation to speak.

$$W_{measure} = \{(t_i, s_i) \in \text{au12_series} \mid t_{stim} + 0.5 \leq t_i \leq t_{stim} + 5.0\}$$

$$W_{baseline} = \{(t_i, s_i) \in \text{au12_series} \mid t_{stim} - 5.0 \leq t_i \leq t_{stim} - 2.0\}$$

STEP 3: P90 AGGREGATION AND GATED REWARD

The 90th percentile is used instead of the mean or maximum to reduce sensitivity to single-frame noise spikes while still capturing the peak smile response. If the measurement window contains zero frames, P90 defaults to 0.0.

The semantic gate *G_t* is binary and comes from the `is_match` output (§8). If the greeting was not delivered, the reward is zero regardless of smile intensity.

$$P_{90} = \text{percentile}_{90}\big(\{s_i \mid (t_i, s_i) \in W_{measure}\}\big)$$

$$G_t = \begin{cases} 1 & \text{if is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$r_t = P_{90} \times G_t$$

STEP 4: FRACTIONAL BETA-BERNOULLI POSTERIOR UPDATE

Each arm maintains a Beta posterior initialized with a uniform prior Beta(1, 1). After each encounter, the posterior is updated with the fractional reward *r_t* ∈ [0, 1]. Unlike standard Bernoulli updates that require binary 0/1 outcomes, the fractional update preserves the continuous gradient of the bounded AU12 signal.

Arm selection uses Thompson Sampling: for each arm, a value is sampled from its current Beta posterior, and the arm with the highest sampled value is selected for the next encounter.

$$\alpha \leftarrow \alpha + r_t, \quad \beta \leftarrow \beta + (1 - r_t)$$

$$a^* = \arg\max_a \theta_a, \quad \theta_a \sim \mathrm{Beta}(\alpha_a, \beta_a)$$

7B.2 REWARD PIPELINE AND THOMPSON SAMPLING — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import random
from dataclasses import dataclass
from typing import Optional
import numpy as np

STIMULUS_WINDOW_START: float = 0.5   # seconds after stimulus
STIMULUS_WINDOW_END: float = 5.0     # seconds after stimulus
BASELINE_WINDOW_START: float = -5.0  # seconds before stimulus
BASELINE_WINDOW_END: float = -2.0    # seconds before stimulus
DEFAULT_PRIOR_ALPHA: float = 1.0
DEFAULT_PRIOR_BETA: float = 1.0


@dataclass
class RewardResult:
    gated_reward: float
    p90_intensity: float
    semantic_gate: int
    n_frames_in_window: int
    baseline_b_neutral: Optional[float]


def extract_stimulus_window(
    au12_series: list[dict], stimulus_time_s: float,
    start_offset: float = STIMULUS_WINDOW_START,
    end_offset: float = STIMULUS_WINDOW_END,
) -> list[dict]:
    t_start = stimulus_time_s + start_offset
    t_end = stimulus_time_s + end_offset
    return [o for o in au12_series if t_start ⩽ o["timestamp_s"] ⩽ t_end]


def compute_reward(
    au12_series: list[dict], stimulus_time_s: float,
    is_match: bool, x_max: Optional[float] = None,
) -> RewardResult:
    window = extract_stimulus_window(au12_series, stimulus_time_s)
    n_frames = len(window)

    if n_frames == 0:
        p90 = 0.0
    else:
        p90 = float(np.percentile([o["intensity"] for o in window], 90))

    if x_max is not None and x_max > 0:
        p90 = min(p90 / x_max, 1.0)

    gate = 1 if is_match else 0
```

```
    gated = p90 * gate

    baseline_obs = extract_stimulus_window(
        au12_series, stimulus_time_s,
        start_offset=BASELINE_WINDOW_START, end_offset=BASELINE_WINDOW_END,
    )
    baseline = (
        float(np.mean([o["intensity"] for o in baseline_obs]))
        if baseline_obs else None
    )

    return RewardResult(
        gated_reward=gated, p90_intensity=p90, semantic_gate=gate,
        n_frames_in_window=n_frames, baseline_b_neutral=baseline,
    )


def thompson_sample_arm(arms: dict[str, tuple[float, float]]) -> str:
    samples = {
        arm_id: random.betavariate(alpha, beta)
        for arm_id, (alpha, beta) in arms.items()
    }
    return max(samples, key=samples.get)


def update_posterior(
    alpha: float, beta: float, reward: float
) -> tuple[float, float]:
    return alpha + reward, beta + (1.0 - reward)
```

7C Physiological co-modulation analytics

This section defines physiological co-modulation as a rolling Pearson correlation between streamer and operator RMSSD time series.

RMSSD snapshots are derived in Module C from normalized physiological chunks and persisted per segment. The result is observational only and does not enter §7B reward computation.

7C.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$R_s^{(k)}$	`streamer_rmssd_bins[k]`	Streamer RMSSD in a 1-minute resampled analysis bin.
$R_o^{(k)}$	`operator_rmssd_bins[k]`	Operator RMSSD in a 1-minute resampled analysis bin.
C_{mod}	`co_modulation_index`	Pearson correlation between aligned streamer and operator RMSSD sequences.
N	`n_paired_observations`	Number of aligned paired observations in the analysis window.
$coverage_ratio$	`coverage_ratio`	Fraction of the analysis window with valid paired observations.
MIN_COMOD_PAIRS	`MIN_COMOD_PAIRS`	Minimum aligned pairs required for a valid correlation.
$COMOD_WINDOW_MINUTES$	`COMOD_WINDOW_MINUTES`	Rolling analysis window duration in minutes.

STEP 1: DATA SOURCE AND ELIGIBILITY

Use per-segment physiological snapshots from `physiology_log`.

A snapshot is eligible only when `rmssd_ms` is not null, `is_valid = TRUE`, and `is_stale = FALSE`.

STEP 2: WINDOWING, RESAMPLING, AND TIME ALIGNMENT

For each subject, select eligible snapshots in the 10-minute window ending at `window_end_utc`.

Resample each series to 1-minute mean bins and inner-join on timestamp.

$$R_s = \text{resample}_{1\text{min}}(\text{streamer_rmssd})$$

$$R_o = \text{resample}_{1\text{min}}(\text{operator_rmssd})$$

STEP 3: PEARSON CORRELATION AND NULL GATING

Compute Pearson correlation on the aligned RMSSD series. Positive values indicate alignment; negative values indicate divergence.

- If N < 4 (MIN_COMOD_PAIRS), co_modulation_index = null.
- If either aligned series has zero variance, co_modulation_index = null.
- If N >= 4 and both aligned series have non-zero variance, compute co_modulation_index using scipy.stats.pearsonr.

$$C_{mod} = \frac{\sum(R_s^{(k)} - \bar{R}_s)(R_o^{(k)} - \bar{R}_o)}{\sqrt{\sum(R_s^{(k)} - \bar{R}_s)^2 \sum(R_o^{(k)} - \bar{R}_o)^2}}$$

STEP 4: COVERAGE RATIO AND PERSISTENCE

Coverage ratio measures the fraction of the window with valid paired observations.

Persist the result even when `co_modulation_index` is null.

- co_modulation_index
- n_paired_observations
- coverage_ratio
- streamer_rmssd_mean
- operator_rmssd_mean
- window_start_utc
- window_end_utc

$$\text{coverage_ratio} = \frac{N}{\max(|R_s|, |R_o|, 1)}$$

STEP 5: SCOPE CONSTRAINT

The Co-Modulation Index is observational and does not modify `compute_reward()` or Thompson Sampling updates.

7C.2 PHYSIOLOGICAL CO-MODULATION — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import pandas as pd
from scipy.stats import pearsonr

MIN_COMOD_PAIRS: int = 4
COMOD_WINDOW_MINUTES: int = 10
RESAMPLE_RULE: str = "1min"


@dataclass(frozen=True)
class CoModulationResult:
    co_modulation_index: Optional[float]
    n_paired_observations: int
    coverage_ratio: float
```

```python
    streamer_rmssd_mean: Optional[float]
    operator_rmssd_mean: Optional[float]


def _resample_rmssd(window_df: pd.DataFrame, subject_role: str) -> pd.Series:
    role_df = (
        window_df.loc[
            window_df["subject_role"] == subject_role,
            ["source_timestamp_utc", "rmssd_ms"],
        ]
        .dropna(subset=["rmssd_ms"])
        .sort_values("source_timestamp_utc")
    )

    if role_df.empty:
        return pd.Series(dtype="float64")

    return (
        role_df.set_index("source_timestamp_utc")["rmssd_ms"]
        .resample(RESAMPLE_RULE)
        .mean()
        .dropna()
    )


def compute_comodulation(
    physiology_log: pd.DataFrame,
    window_end_utc: pd.Timestamp,
) -> CoModulationResult:
    window_end_utc = pd.Timestamp(window_end_utc, tz="UTC")
    window_start_utc = window_end_utc - pd.Timedelta(minutes=COMOD_WINDOW_MINUTES)

    eligible = physiology_log.copy()
    eligible["source_timestamp_utc"] = pd.to_datetime(
        eligible["source_timestamp_utc"], utc=True
    )
    eligible = eligible.loc[
        (eligible["source_timestamp_utc"] > window_start_utc)
        & (eligible["source_timestamp_utc"] <= window_end_utc)
        & eligible["rmssd_ms"].notna()
        & (eligible["is_valid"] == True)
        & (eligible["is_stale"] == False)
    ]

    streamer = _resample_rmssd(eligible, "streamer")
    operator = _resample_rmssd(eligible, "operator")

    aligned = pd.concat(
        [streamer.rename("streamer"), operator.rename("operator")],
        axis=1,
        join="inner",
    ).dropna()

    n_pairs = int(len(aligned))
    coverage_ratio = float(
        n_pairs / max(len(streamer), len(operator), 1)
    )

    streamer_mean = float(streamer.mean()) if not streamer.empty else None
    operator_mean = float(operator.mean()) if not operator.empty else None
```

```
    if n_pairs < MIN_COMOD_PAIRS:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    if aligned["streamer"].nunique() < 2 or aligned["operator"].nunique() < 2:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    co_modulation_index, _p_value = pearsonr(
        aligned["streamer"], aligned["operator"]
    )

    return CoModulationResult(
        co_modulation_index=float(co_modulation_index),
        n_paired_observations=n_pairs,
        coverage_ratio=coverage_ratio,
        streamer_rmssd_mean=streamer_mean,
        operator_rmssd_mean=operator_mean,
    )
```

SECTION 8

Deterministic LLM prompt specification

Module D must determine whether a spoken utterance satisfies a predefined semantic rule such as a required greeting phrase. Traditional string-matching algorithms like Levenshtein distance or BLEU fail because human speech contains fillers, paraphrasing, and transcription noise.

To perform robust evaluation while maintaining reproducibility, LSIE-MLF uses Azure OpenAI Chat Completion models configured in deterministic mode. Temperature is fixed to zero, nucleus sampling is disabled, and structured JSON outputs are enforced through schema constraints.

The `is_match` boolean is the reward gate $G_t \in \{0, 1\}$. When `is_match` is True, P90 AU12 intensity contributes to the posterior update; otherwise the reward is zero. `confidence_score` is persisted for offline analysis only and does not modulate the reward.

8.1 Inference parameter constraints

PARAMETER	VALUE	JUSTIFICATION
temperature	0.0	Collapses the probability distribution to deterministic token selection.
top_p	1.0	Disables nucleus sampling ensuring deterministic token selection.
max_tokens	500	Ensures adequate space for reasoning plus structured JSON output.
response_format	JSON Schema Structured Outputs	Forces the model to return machine-parseable JSON rather than conversational text.
seed	42	Activates Azure OpenAI deterministic inference mode.

8.2 Expected output schema

greeting_eval.schema.json JSON

```
{
 "type": "object",
 "required": [
  "reasoning",
  "is_match",
  "confidence_score"
 ],
 "properties": {
  "reasoning": {
   "type": "string"
  },
  "is_match": {
   "type": "boolean"
  },
  "confidence_score": {
   "type": "number",
   "minimum": 0,
   "maximum": 1
  }
 },
 "additionalProperties": false
}
```

8.3 System prompt (canonical)

system_prompt.txt TEXT

```text
You are a highly precise linguistic evaluation assistant.
Your objective is to determine whether a transcribed 'Actual Utterance' is semantically
↪  equivalent to an 'Expected Greeting Rule'.

Inputs:
1. Expected Greeting Rule
2. Actual Utterance

Evaluation constraints:
- Minor transcription errors and filler words are acceptable if the semantic intent remains
↪  unchanged.
- Major deviations in subject matter must be rejected.

Steps:
1. Produce a reasoning explanation evaluating semantic similarity.
2. Output boolean is_match.
3. Output confidence_score between 0.00 and 1.00.

Return the result strictly as valid JSON without markdown formatting.
```

SECTION 9

Docker compose runtime topology

The LSIE-MLF runtime environment consists of multiple containerized services orchestrated through Docker Compose. Containers share a common bridge network and specific named volumes enabling high-throughput IPC communication and persistent storage where required.

9.1 Container topology

CONTAINER	ROLE	IMAGE	CONFIGURATION
`api`	API Server providing FastAPI REST endpoints and physiological webhook ingress for Oura webhooks.	`python:3.11-slim`	Port: 8000:8000 Net: appnetwork GPU: None Restart: unless-stopped
`worker`	ML Worker executing Celery inference tasks.	`nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04`	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: on-failure:5

CONTAINER	ROLE	IMAGE	CONFIGURATION
redis	Message Broker used by Celery.	redis:7-alpine	Port: 6379 (internal) Net: appnetwork GPU: None Restart: unless-stopped
postgres	Persistent Store for experiment and metric data.	postgres:16-alpine	Port: 5432 (internal) Net: appnetwork GPU: None Restart: unless-stopped
stream_scrcpy	Capture Container extracting raw USB audio/video streams.	ubuntu:24.04 + scrcpy v3.3.4	Port: None Net: appnetwork GPU: None Restart: on-failure:3
orchestrator	Module C orchestrator assembling 30-second segments, processing AU12 frames, and dispatching Celery tasks.	nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04	Port: None Net: appnetwork GPU: All (capabilities: [gpu], driver: nvidia) Restart: on-failure:5

9.2 Volume mounts

VOLUME	MOUNT PATH	CONTAINERS	PURPOSE
ipc-share	/tmp/ipc/	stream_scrcpy, worker, orchestrator	Shared volume containing IPC named pipes.
data-raw	/data/raw/	worker, orchestrator	Transient debug media buffers.
data-interim	/data/interim/	worker, orchestrator	Intermediate processing artifacts.
data-processed	/data/processed/	worker, orchestrator, api	Validated inference outputs and fallback storage.
pg-data	/var/lib/postgresql/data/	postgres	Persistent PostgreSQL database storage.

9.3 GPU allocation

The ML Worker and orchestrator containers receive GPU access through Docker's `deploy.resources` reservation syntax and require CUDA 12 with cuDNN 8 for CTranslate2 compatibility. cuDNN 8 preserves Pascal SM 6.1 dp4a compatibility required by the target hardware.

GPU allocation YAML

```yaml
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: all
          capabilities: [gpu]
```

9.4 Device passthrough

USB passthrough allows the capture container to communicate directly with the Android device through the host kernel device subsystem.

Device passthrough YAML

```yaml
devices:
  - /dev/bus/usb:/dev/bus/usb
```

9.5 Network configuration

All services run within the same Docker bridge network allowing container name DNS resolution. Only the API service exposes ports externally.

9.6 Service dependency order

1. redis — starts first; health check redis-cli ping.
2. postgres — starts after redis; health check pg_isready.
3. stream_scrcpy — starts after redis to initialize IPC pipe.
4. worker — starts after redis, postgres, and stream_scrcpy.
5. orchestrator — starts alongside worker after redis, postgres, and stream_scrcpy.
6. api — starts after worker and postgres.

SECTION 10

Dependency and runtime matrix

10.1 System requirements

COMPONENT	REQUIRED VER	NOTES
Python	3.11.x	All containers MUST run Python 3.11. Python 3.12+ is untested with CTranslate2.
NVIDIA CUDA	⩾12.0	Required for faster-whisper and CTranslate2 runtime compatibility.
cuDNN	⩾8.0	Bundled within nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04; cuDNN 8 preserves Pascal SM 6.1 dp4a compatibility required by the hardening baseline.
NVIDIA Container Toolkit	latest	Required for Docker GPU passthrough.
GPU VRAM	⩾6GB	Minimum VRAM for faster-whisper large-v3 INT8 plus MediaPipe runtime.
Docker Compose	v2.x	Required for deploy.resources GPU reservation syntax.
scrcpy	3.3.4	Required for raw Android USB audio/video extraction; v3.3.4 provides the –no-playback flag and dual-instance –port range support used by the capture entrypoint.

10.2 Python package pinning

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
faster-whisper	1.2.1	worker	GPU accelerated speech transcription
mediapipe	0.10.9	worker	Face Mesh 478 landmark inference
praat-parselmouth	0.4.4	worker	Praat acoustic feature extraction
spacy	3.7.x	worker	NLP preprocessing pipeline
psycopg2-binary	2.9.x	worker, api	PostgreSQL driver
pandas	2.2.x	worker	Time-series data processing
scipy	1.13.*	worker	Statistical computation
celery	5.4.x	worker, api	Distributed task queue
redis	5.x	worker, api	Celery broker transport
fastapi	0.110.x	api	REST API framework
uvicorn	0.29.x	api	ASGI server runtime
pydantic	2.x	worker, api	Schema validation
numpy	1.26.x	worker	Vectorized AU12 calculations
pycryptodome	⩾3.20.0	worker	AES-256-GCM encryption
patchright	1.58.2	worker	Stealth browser automation

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
`TikTokLive`	`5.0.x`	worker	TikTok WebSocket ingestion
`PySide6`	`⩾6.11`	N/A (host-installed; not containerized)	Native Qt framework for scripts/debug_studio.py and services/operator_console/

SECTION 11

Variable extraction matrix

Each analytical variable produced within LSIE-MLF is traceable to its source module, data type, storage location, and extraction methodology.

VARIABLE PROFILE	EXTRACTION & ROUTING
Raw Video Frames `byte stream (H.264)` Module A → Transient (kernel buffer)	scrcpy USB mirroring; H.264 stream via `/dev/bus/usb` passthrough.
Raw Audio PCM `byte stream (s16le)` Unit: 48 kHz mono Module A → Transient IPC pipe buffer	scrcpy raw audio capture; s16le 48 kHz mono to IPC Pipe.
Live Event Payloads `dict` Module B → Transient deque buffer	TikTokLive WebSocket ingestion; parsed protobuf event dictionaries.
Action Combo Trigger `bool` Unit: True/False Module B → Persistent Store events table	Composite temporal event correlation across concurrent chat actions.
UTC Drift Offset `float` Unit: seconds Module C → Transient variable	ADB epoch polling every 30s; `drift_offset` = host UTC − Android epoch.
Resampled Audio Chunks `byte stream (s16le)` Unit: 16 kHz mono Module C → Transient stdout pipe	FFmpeg resampling subprocess; 48 kHz → 16 kHz mono s16le on `pipe:1`.

VARIABLE PROFILE	EXTRACTION & ROUTING
3D Facial Landmarks `np.ndarray (478,3)` Unit: normalized coordinates Module D → Transient memory	MediaPipe Face Mesh; 478-vertex 3D landmark array per frame.
AU12 Intensity Score `float` Unit: 0.0–5.0 (compute_intensity) or 0.0–1.0 (compute_bounded_-intensity for Thompson Sampling reward pipeline) Module D → Persistent Store metrics table	AU12Normalizer (§7A); IOD-normalized lip-corner distance with neutral baseline subtraction. Exposed as 0.0–5.0 (legacy) and 0.0–1.0 (bounded, for reward pipeline).
Vocal Pitch F0 `float` Unit: 75–600 Hz Module D → Persistent Store metrics table	Praat via parselmouth; fundamental frequency extraction from speech audio.
Jitter `float` Unit: 0–1 ratio Module D → Persistent Store metrics table	Praat via parselmouth; cycle-to-cycle pitch perturbation ratio.
Shimmer `float` Unit: 0–1 ratio Module D → Persistent Store metrics table	Praat via parselmouth; cycle-to-cycle amplitude perturbation ratio.
ASR Transcription `string` Unit: UTF-8 text Module D → Persistent Store transcripts table	faster-whisper with CTranslate2 INT8 backend; segment-level speech-to-text.
Semantic Match `JSON object` Unit: confidence 0–1 Module D → Persistent Store evaluation table	Azure OpenAI deterministic evaluation (§8); `is_match`, `confidence_score`, `reasoning`.
Evaluation Variance `float` Unit: 0–infinity Module E → Persistent Store experiments table	Thompson Sampling posterior statistics; variance of Beta distribution per arm.

VARIABLE PROFILE	EXTRACTION & ROUTING
External Context Metadata `JSON dict` Module F → Persistent Store context table	patchright headless browser scraping; creator and trending topic metadata.
gated_reward `float` Unit: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); r_t = `P90` × `semantic_gate`.
p90_intensity `float` Unit: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); 90th percentile of bounded AU12 in stimulus window.
semantic_gate `int` Unit: {0, 1} Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); binary gate from `is_match`: 1 if True, 0 otherwise.
n_frames_in_window `int` Unit: [0, +infinity) Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); count of AU12 observations in stimulus window.
baseline_b_neutral `float | null` Unit: [0.0, +infinity) or null Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); mean bounded AU12 in pre-stimulus window, or null.
stimulus_time `float` Unit: UTC epoch seconds Module C → Transient variable	Drift-corrected UTC epoch of stimulus injection; defines baseline and measurement windows for `compute_reward()` (§7B).
RMSSD (Streamer) `float | null` Unit: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
RMSSD (Operator) `float | null` Unit: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.

VARIABLE PROFILE	EXTRACTION & ROUTING
Heart Rate (Streamer/Operator) `int | null` Unit: 20–300 BPM or null Module C → Persistent Store physiology_log table	Rounded mean of overlapping `heart_rate_items_bpm` values in the trailing derivation window (§4.C.4); null when no heart-rate values exist.
Physiological Validity Ratio `float` Unit: [0.0, 1.0] Module C → Persistent Store physiology_log table	`validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` over the trailing derivation window (§4.C.4).
Physiological Validity Flag `bool` Unit: True / False Module C → Persistent Store physiology_log table	`is_valid = (validity_ratio ⩾ PHYSIO_VALIDITY_MIN)`; default threshold 0.80 (§4.C.4).
Physiological Source Kind `string` Unit: {'ibi', 'session'} Module C → Persistent Store physiology_log table	ibi when the snapshot is derived from interbeat-interval chunks; otherwise session when derived from provider-session HRV chunks (§4.C.4).
Physiological Derivation Method `string` Unit: {'provider', 'server'} Module C → Persistent Store physiology_log table	`server` for IBI-derived RMSSD; `provider` for provider-session HRV snapshots unless later recomputed (§4.C.4).
Physiological Window Length `int` Unit: seconds, > 0 Module C → Persistent Store physiology_log table	Length of the trailing derivation window; default `PHYSIO_DERIVE_WINDOW_S = 300` (§4.C.4).
Physiological Freshness `float` Unit: seconds, >= 0 Module C → Persistent Store physiology_log table	Computed at segment assembly: `wall_clock - source_timestamp_utc` (§4.C.4).
Physiological Staleness Flag `bool` Unit: True / False Module C → Persistent Store physiology_log table	`freshness_s > PHYSIO_STALENESS_THRESHOLD_S` (default 600s) (§4.C.4).
Co-Modulation Index `float | null` Unit: [-1.0, +1.0] or null Module E → Persistent Store co-modulation_log table	Pearson correlation on 1-minute resampled, time-aligned valid streamer/operator RMSSD sequences over a 10-minute rolling window (§7C).

VARIABLE PROFILE	EXTRACTION & ROUTING
Physiological Coverage Ratio `float` Unit: [0.0, 1.0] Module E → Persistent Store co-modulation_log table	Fraction of 1-minute analysis bins with valid paired RMSSD observations over the 10-minute co-modulation window (§7C).

SECTION 12

Error handling and recovery specification

LSIE-MLF defines four standardized failure categories. Each module must implement deterministic recovery behavior for each category. Physiological webhook rejections such as invalid HMAC signatures, malformed payloads, duplicate suppression, and Redis enqueue retries are application-level guardrails, not separate failure categories. Physiological staleness is a data-quality condition: Module C marks `is_stale = true` without blocking segment dispatch, and Module E returns a null co-modulation index when paired non-stale observations are insufficient.

12.1 Network disconnection

MODULE	BEHAVIOR
Module A	N/A
Module B	Exponential WebSocket reconnection up to 10 retries; physiological webhook ingress is handled by the API Server, and upstream transport failures rely on provider-managed retries after timeout or HTTP 503 responses
Module C	N/A
Module D	Retry Azure OpenAI request once then record null
Module E	Buffer database writes and retry every 5s
Module F	Return empty scrape result

12.2 Hardware device loss

MODULE	BEHAVIOR
Module A	USB reconnection polling every 2s for 60s then container restart
Module B	N/A

MODULE	BEHAVIOR
Module C	Freeze drift offset after 3 consecutive ADB poll failures (`DRIFT_FREEZE_AFTER_FAILURES` = 3). While frozen, return the cached offset without additional ADB calls; reset drift to zero after 300 seconds / 5 minutes of frozen state (`DRIFT_RESET_TIMEOUT` = 300).
Module D	Terminate worker container if GPU unavailable
Module E	N/A
Module F	N/A

12.3 Worker process crash

MODULE	BEHAVIOR
Module A	Pipe overflow handled via EAGAIN non-blocking writes
Module B	Reconnect WebSocket on restart
Module C	Restart FFmpeg within 1 second when AudioResampler detects a dead subprocess or EOF. If the video capture thread dies after an IPC / PyAV pipe break, the run loop recreates and restarts `VideoCapture` so the upstream scrcpy path can emit a fresh MKV header.
Module D	Docker restart policy on-failure:5
Module E	Log Celery task failure and continue
Module F	Retry scraping task up to two times

12.4 Queue overload

MODULE	BEHAVIOR
Module A	Kernel pipe overflow discards data silently
Module B	Deque ring buffer evicts oldest entries; physiological event enqueue uses Redis list `physio:events`, and enqueue backpressure returns HTTP 503 so the provider retries instead of silently dropping events
Module C	Deque maxlen eviction policy; bounded physiological drain (max 100 events per cycle) prevents stale or bursty webhook traffic from stalling 30-second segment assembly, and malformed physiological events are logged and discarded
Module D	Celery queue buffers tasks in Redis
Module E	Overflow records written to CSV fallback storage; when physiological coverage is incomplete or non-stale paired samples are insufficient, Module E records a null co-modulation result rather than failing the analytics pipeline
Module F	Limit concurrent scraping tasks

SECTION 13

Autonomous implementation audit checklist

Before deployment the system must satisfy every verification item defined below to ensure deterministic implementation.

#	AUDIT ITEM	VERIFICATION CRITERION
1	Directory structure	Repository directories match the hierarchy defined in Section 3.
2	Docker topology	Six containers are defined with correct images, networks, volumes, and restart policies.
3	IPC lifecycle	Named pipe lifecycle implemented according to Section 4.A.2.
4	Audio pipeline	FFmpeg resampling from 48 kHz to 16 kHz executed correctly.
5	Drift correction	ADB polling every 30 seconds with `drift_offset` applied.
6	AU12 implementation	Landmark indices and epsilon guard implemented correctly.
7	LLM determinism	Azure OpenAI configured with temperature 0 and structured outputs.
8	Ephemeral Vault	AES-256-GCM encryption and 24-hour secure deletion enforced.
9	Schema validation	InferenceHandoffPayload validated via Pydantic.
10	Module contracts	Inputs, outputs, dependencies, side effects, and failures implemented exactly as specified.
11	Error handling	All four failure categories implemented across modules.
12	Dependency versions	Pinned runtime versions match the dependency matrix.
13	Variable traceability	All variables defined in the extraction matrix are produced by the correct module.
14	Data classification	Transient, debug, and permanent storage tiers enforced.
15	Canonical terminology	Only canonical component names are used throughout code and configuration.
16	Reward pipeline	Thompson Sampling reward pipeline implements the mathematical recipe from §7B: P90 aggregation, binary semantic gate, fractional Beta-Bernoulli posterior updates with Beta(1,1) prior, and stimulus window boundaries [t+0.5s, t+5.0s].
17	Physiological schema compatibility	`_physiological_context` is optional on InferenceHandoffPayload. Payloads without physiological data remain valid. The field `physiological_sample_event_schema` contains the PhysiologicalChunkEvent definition.
18	Physiological freshness gating	is_stale is True when freshness_s exceeds PHYSIO_STALENESS_THRESHOLD_S. Stale samples do not block segment dispatch. Module E excludes stale samples from co-modulation computation.

#	AUDIT ITEM	VERIFICATION CRITERION
19	Physiological data governance	No raw Oura webhook JSON payloads reach the Persistent Store. Only normalized scalar derivatives and metadata (RMSSD, HR, validity_ratio, freshness, source_kind, derivation_method, co-modulation index) are persisted to physiology_log and comodulation_log tables.
20	Co-modulation determinism	Co-modulation analytics produce null when fewer than 4 paired 1-minute aligned observations exist or either aligned series has zero variance. Pearson correlation uses scipy.stats.pearsonr. Results are deterministic for the same input data.
21	Reward pipeline invariance	`compute_reward()` and `ThompsonSamplingEngine.update()` follow §7B exactly. Physiological analytics do not enter the reward computation.

— END OF SPECIFICATION —